Trabalho Prático Sistemas Operativos

Plataforma de Gestão e Serviço de Táxis

Guilherme Marques Ferreira Nº 2024133874
Bernardo Alcântara Vasconcelos Nº 2024128419

isec
Politécnico de Coimbra

Índice

1. Introducão
2. Arquitetura do Sistema
3. Mecanismos de Comunicação
4. Descricão dos Programas
5. Estruturas de Dados
6. Sincronização e Concorrência
7. Gestão do Tempo Simulado
8. Compilação e Execução
9. Tabela de Funcionalidades
10. Conclusão

1. Introducão

Este relatório descreve a implementação de uma plataforma de gestão de um serviço de táxi composto por uma frota de veículos autónomos. O sistema foi desenvolvido em linguagem C para ambiente UNIX, utilizando os mecanismos de comunicação inter-processos.

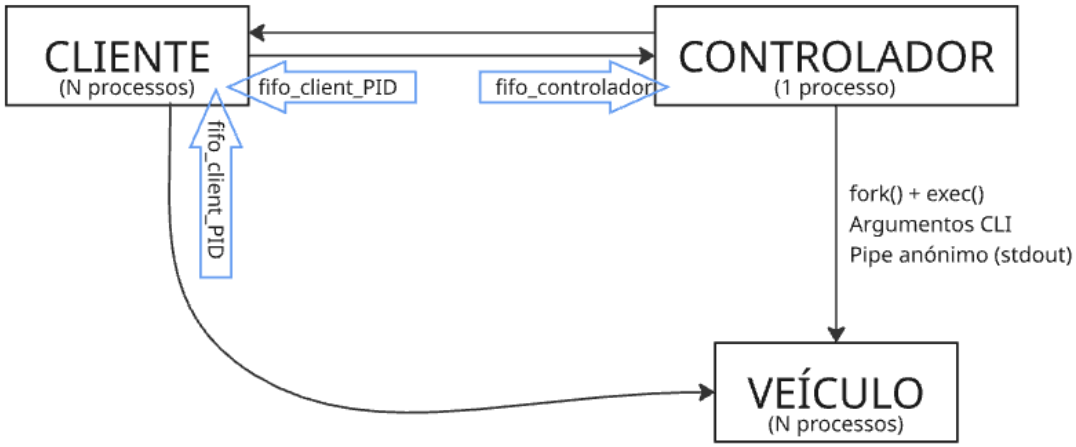
A plataforma é composta por três aplicações distintas:

- **Cliente**: Interface para utilizadores agendarem serviços de transporte.
- **Controlador**: Servidor central que gere a frota e os pedidos.
- **Veículo**: Simula o funcionamento de um veículo autónomo.

O trabalho foca-se no uso correto de chamadas ao sistema operativo, named pipes (FIFOs), pipes anonimos, processos, threads e sinais.

2. Arquitetura do Sistema

2.1 Visão Geral



2.2 Fluxo de Comunicação

1. **Cliente -> Controlador**: Named pipe fifo_controlador (escrita)
2. **Controlador -> Cliente**: Named pipe /tmp/fifo_client_<PID> (escrita)
3. **Controlador -> Veículo**: Argumentos da linha de comandos (via execl())
4. **Veículo -> Controlador**: Pipe anónimo (stdout redirecionado via dup2())
5. **Veículo -> Cliente**: Named pipe /tmp/fifo_client_<PID> (escrita direta)

3. Mecanismos de Comunicação

3.1 Named Pipes (FIFOs)

Os named pipes são utilizados para comunicação bidirecional entre processos independentes:

FIFO do Servidor (fifo_controlador): Criado pelo controlador no início da execução - Todos os clientes escrevem pedidos neste FIFO - O controlador lê de forma não-bloqueante (O_NONBLOCK).

FIFO do Cliente (/tmp/fifo_client_<PID>): Criado por cada processo do cliente - Usado para receber respostas do controlador e notificações do veículo - O PID é usado para garantir individualidade do nome.

Implementação (cliente.c, linhas 14-23):

```
void enviar_server(const char *msg){
  int fd = open(SERVER_FIFO, O_WRONLY | O_NONBLOCK);
  if(fd<0){
    perror("Erro ao abrir fifo do servidor");
    return;
  }
  write(fd, msg, strlen(msg));
  write(fd, "\n", 1);
  close(fd);
}
```

3.2 Pipes Anónimos

Utilizados para capturar a telemetria do veículo (stdout) pelo controlador:

Implementação (controlador.c, linhas 400-464):

```
int pfd[2];
pipe(pfd);  // Cria pipe anónimo

pid_t pid = fork();
if(pid == 0){
  // FILHO (veiculo): redireciona stdout para pfd[1]
  close(pfd[0]);
  dup2(pfd[1], STDOUT_FILENO);
  close(pfd[1]);
  execl("./veiculo", ...);
} else {
  // PAI (controlador): le de pfd[0]
  close(pfd[1]);
  // Cria thread para ler telemetria
}
```

3.3 Sinais

SIGINT (Ctrl+C): - Capturado pelo controlador para terminar o sistema de forma ordenada - Capturado pelo cliente para terminação graceful

SIGTERM: - Enviado pelo controlador aos clientes na terminação do sistema

SIGUSR1: - Enviado pelo controlador ao veículo para cancelar um serviço em curso - O veiculo captura este sinal e termina a viagem imediatamente

Implementação (veiculo.c, linhas 14-17):

```
void sigusr1_handler(int s){
  (void)s;
  cancelar = 1;  // Flag global para interromper o loop
}
```

4. Descrição dos Programas

4.1 Cliente (cliente.c)

Função: Interface de utilizador para interação com a plataforma.

Execução: ./cliente <username>

Comandos disponíveis:

Comando	Descrição
agendar <hora> <local> <distancia>	Agenda um serviço de transporte
consultar	Lista os serviços agendados pelo utilizador
cancelar <id>	Cancela um serviço (id=0 cancela todos)
terminar	Sai da aplicação

Multiplexagem de I/O: O cliente utiliza select() para monitorizar simultaneamente: - stdin (comandos do utilizador) - FIFO do cliente (mensagens do controlador/veiculo)

```
while(!terminar_cliente){
  FD_ZERO(&read_fds);
  FD_SET(STDIN_FD, &read_fds);
  FD_SET(fd_fifo, &read_fds);

  select(max_fd + 1, &read_fds, NULL, NULL, NULL);

  if(FD_ISSET(fd_fifo, &read_fds)){
    // Processa mensagens do FIFO
  }
  if(FD_ISSET(STDIN_FD, &read_fds)){
    // Processa comandos do utilizador
  }
}
```

Protocolo de Mensagens (Cliente -> Controlador):

- REGISTER <user> <client_fifo> - Registo inicial

- AGENDAR <user> <hora> <local> <dist> <client_fifo> - Agendar serviço

- CONSULTAR <user> <client_fifo> - Consultar serviços

- CANCELAR <user> <id> <client_fifo> - Cancelar serviço

4.2 Controlador (controlador.c)

Função: Servidor central que gere toda a plataforma.

Execução: ./controlador

Configuração: Variável de ambiente NVEICULOS define o tamanho da frota (max 10).

Comandos do Administrador:

| Comando | Descrição |

| listar | Mostra todos os serviços agendados |

| utiliz | Lista utilizadores conectados |

| frota | Mostra estado dos veículos em execução |

| cancelar <id> | Cancela serviço (id=0 cancela todos) |

| km | Total de quilómetros percorridos |

| hora | Hora simulada atual |

| terminar | Encerra o sistema |

Arquitetura Multi-threaded:

1. **Thread Principal**: Loop que:
   - Lê pedidos do FIFO do servidor
   - Incrementa o tempo simulado (1 segundo real = 1 unidade)
   - Verifica serviços agendados para lançar
   - Recolhe processos filho terminados (waitpid com WNOHANG)
2. **Thread Admin**: Lê comandos do administrador via stdin (bloqueante)
3. **Threads de Telemetria**: Uma por veículo lançado, lê o stdout via pipe anónimo

Lançamento de Veículos (controlador.c, linhas 387-465):

```
void launch_vehicle_by_index(int idx){
  // 1. Cria pipe anonimo
  pipe(pfd);

  // 2. Fork
  pid = fork();

  if(pid == 0){
    // Filho: redireciona stdout e executa veiculo
    dup2(pfd[1], STDOUT_FILENO);
    execl("./veiculo", "veiculo", user, client_fifo, dist, id, NULL);
  } else {
    // Pai: cria thread para ler telemetria
    pthread_create(&th, NULL, telemetry_thread, args);
  }
}
```

4.3 Veículo (veiculo.c)

Função: Simula um veículo autónomo em serviço.

Execução: ./veiculo <user> <client_fifo> <distancia> <id> (Lançado automaticamente pelo controlador)

Fluxo de Execução:

1. Recebe parâmetros via argumentos da linha de comandos

2. Notifica o cliente que chegou (VEICULO_CHEGOU)

3. Inicia viagem imediatamente (simplificação)

4. Reporta progresso a cada km percorrido (PROGRESS x/total)

5. No final, reporta conclusão ou cancelamento

Telemetria (stdout -> Controlador):

- ARRIVED <id> - Veículo chegou ao local

- CLIENT_ENTERED <id> - Cliente entrou

- PROGRESS <percorridos>/<total> - Progresso da viagem

- END_COMPLETED <km> - Viagem concluída

- END_CANCELLED <km> - Viagem cancelada

Comunicação com Cliente (FIFO):

- VEICULO_CHEGOU <id> - Notifica chegada

- VEICULO_CANCELADO <id> <km> - Notifica cancelamento - Mensagem de conclusão com km percorridos

5. Estruturas de Dados

5.1 Serviço (controlador.h)

```
typedef enum {
  S_AGENDADO,  // 0 - Servico agendado
  S_EM_CURSO,  // 1 - Veiculo em movimento
  S_CANCELADO, // 2 - Cancelado
  S_CONCLUIDO  // 3 - Viagem terminada
} estado_serv;

typedef struct {
  int id;           // ID unico do servico
  char user[64];       // Username do cliente
  char client_fifo[128]; // FIFO para respostas
  int hora;          // Hora agendada (tempo simulado)
  char local[64];      // Local de partida
  int distancia;       // Distancia em km
  estado_serv estado;    // Estado atual
  pid_t pid_veiculo;     // PID do processo veiculo (ou 0)
} servico;
```

5.2 Utilizador Ligado (controlador.h)

```
typedef struct {
  char client_fifo[128]; // FIFO do cliente
  int ativo;         // 1=ativo, 0=inativo
} utilizador_ligado;
```

5.3 Limites do Sistema

Constante	Valor	Descrição
MAX_USERS	30	Maximo de utilizadores
MAX_SERV	200	Maximo de serviços
MAX_VEIC	10	Maximo de veículos simultâneos
LINEBUF	512	Tamanho do buffer de linha

6. Sincronização e Concorrência

6.1 Mutexes Utilizados

O controlador utiliza tres mutexes para proteger recursos partilhados:

```
static pthread_mutex_t serv_mutex;  // Protege array de servicos
static pthread_mutex_t print_mutex; // Protege escrita na consola
static pthread_mutex_t utls_mutex;  // Protege lista de utilizadores
static pthread_mutex_t prog_mutex;  // Protege array de progresso
```

6.2 Função Thread-Safe para Output

```
void safe_printf(const char *fmt, ...) {
  va_list ap;
  pthread_mutex_lock(&print_mutex);
  va_start(ap, fmt);
  vprintf(fmt, ap);
  va_end(ap);
  fflush(stdout);
  pthread_mutex_unlock(&print_mutex);
}
```

6.3 Recolha de Processos Filho

O controlador nao bloqueia a esperar pelo término dos veículos. Utiliza waitpid com WNOHANG:

```
while((w = waitpid(-1, &status, WNOHANG)) > 0){
  // Atualiza estado do servico correspondente
  // Decrementa contador de veiculos em execucao
}
```

7. Gestão do Tempo Simulado

O sistema utiliza tempo simulado em vez do relógio do sistema:

- **Início**: simulated_time = 1 quando o controlador arranca
- **Incremento**: A cada segundo real, incrementa 1 unidade
- **Agendamento**: Serviços sao lançados quando simulated_time >= hora_agendada
- **Velocidade dos Veículos**: 1 km por unidade de tempo

```
// Loop principal do controlador
while(!terminar_flag){
  // ... processar pedidos ...

  sleep(1);  // 1 segundo real
  pthread_mutex_lock(&serv_mutex);
  simulated_time++;
  pthread_mutex_unlock(&serv_mutex);

  check_launch_scheduled();  // Verifica servicos a lancar
}
```

8. Compilação e Execução

8.1 Makefile

O projeto inclui um Makefile com os seguintes targets:

```
CC = gcc
CFLAGS = -Wall -O2
LDFLAGS_CONTROLLER = -pthread

all: controlador cliente veiculo

controlador: controlador.c controlador.h
  $(CC) $(CFLAGS) $< -o $@ $(LDFLAGS_CONTROLLER)

cliente: cliente.c cliente.h
  $(CC) $(CFLAGS) $< -o $@

veiculo: veiculo.c veiculo.h
  $(CC) $(CFLAGS) $< -o $@

clean:
  rm -f controlador cliente veiculo *.o
```

8.2 Compilação

```
# Compilar todos os programas
make all

# Compilar individualmente
make controlador
make cliente
make veiculo

# Limpar ficheiros
make clean
```

8.3 Execução

Terminal 1 - Controlador:

```
export NVEICULOS=5  # Opcional: define tamanho da frota
./controlador
```

Terminal 2 - Cliente:

```
./cliente pedro
```

Terminal 3 - Outro Cliente:

```
./cliente maria
```

8.4 Exemplo de Utilização

Terminal do Cliente (pedro)
CLI> *agendar 10 Coimbra 5*
[SISTEMA] Servico ID 1 agendado com sucesso.
CLI> *consultar*
[Consulta]: SERV id=1 hora=10 local=Coimbra dist=5 estado=0

Apos hora=10, o veiculo é lançado automaticamente
*** [ALERTA] Veiculo ID 1 chegou! Viagem iniciada. ***

Terminal do Controlador
ADM> *frota*
[SISTEMA] Frota em execucao (ID | User | Progresso):
[id:1] user:pedro | progresso:40% | pid:12345
ADM> *km*
[SISTEMA] Total de Km percorridos: 5km

9. Tabela de Funcionalidades

Funcionalidade	Estado	Observações
Cliente		
Registo de utilizador	Cumprido	Username único verificado
Agendar serviço	Cumprido	hora/local/distância
Consultar serviços	Cumprido	Lista serviços do utilizador
Cancelar serviço	Cumprido	Por ID ou todos (id=0)
Terminar cliente	Cumprido	Cancela serviços pendentes
Receber notificações	Cumprido	Via FIFO assincrono
Controlador		
Listar serviços	Cumprido	Comando listar
Listar utilizadores	Cumprido	Comando utiliz
Estado da frota	Cumprido	Comando frota com %
Cancelar serviços	Cumprido	Por ID ou todos
Total de km	Cumprido	Comando km
Hora simulada	Cumprido	Comando hora
Terminar sistema	Cumprido	Notifica clientes
Lancamento automático	Cumprido	Na hora agendada
Limite de veículos	Cumprido	Via NVEICULOS
Veículo		

Funcionalidade	Estado	Observações
Receber args CLI	Cumprido	user/fifo/dist/id
Telemetria stdout	Cumprido	PROGRESS x/total
Notificar cliente	Cumprido	VEICULO_CHEGOU
Cancelamento SIGUSR1	Cumprido	Termina imediatamente
Comunicação		
Named pipes	Cumprido	Cliente <-> Controlador/Veiculo
Pipes anonimos	Cumprido	Veiculo -> Controlador
Sinais	Cumprido	SIGINT/SIGTERM/SIGUSR1
Sincronização		
Mutexes	Cumprido	Proteção de dados partilhados
Thread admin	Cumprido	Comandos não-bloqueantes
Threads telemetria	Cumprido	Uma por veiculo

10. Conclusão

O trabalho implementa com sucesso uma plataforma de gestao de serviço de taxi autónomo, demonstrando o uso correto dos mecanismos de comunicação inter-processos em UNIX:

Pontos Fortes: - Arquitetura clara com separação de responsabilidades

- Uso adequado de named pipes para comunicação cliente-servidor

- Redirecionamento de stdout via pipes anónimos para telemetria

- Sincronização correta com mutexes em ambiente multi-threaded

- Gestão de sinais para cancelamento e terminação

Decisões de Implementação: - O cliente utiliza select() para multiplexagem de I/O (permitido no cliente)

- O controlador utiliza threads para não bloquear no input do admin

- Cada veículo e um processo independente lançado via fork() + execl()

- O tempo simulado permite testes mais rápidos que tempo real